DuocUC

ACREDITTIA ADMINISTRACIÓN DEL NEGOCIO (BACKOFFICE SAAS)

Integrantes:
Angel Morales — Scrum Master y Data Engineer
Paula Molina — Backend Developer (REST API)
Antonia Valencia — Backend Developer y QA
Asignatura: CAPSTONE_300D

Docente: Claudia Reinoso
Repositorio GitHub:
https://github.com/AngelMorales-1231/Capstone-Grupo5.git

ABSTRACT

RESUMEN EJECUTIVO/ EXECUTIVE SUMMARY

El proyecto Acredittia, desarrollado por el equipo DADN_04, busca centralizar la gestión operativa y comercial de más de 1.142 empresas contratistas. Durante la Fase 2, se completaron los entregables del Hito 1 y las principales rutas operativas del Hito 2. Entre los avances destacan la implementación de 20 endpoints administrativos, el modelamiento de la base de datos con SQLAlchemy y PostgreSQL, los controles de acceso y auditoría, y el despliegue mediante Docker, Vercel y Neon. El trabajo se organiza mediante Scrum, con sprints de dos semanas y control de versiones en GitHub.

The Acredittia project, developed by the DADN_04 team, aims to centralize operational and commercial management for over 1,142 contractor companies. During Phase 2, Milestone 1 deliverables and the main operational routes for Milestone 2 were completed. Key achievements include the implementation of 20 administrative endpoints, relational database modeling with SQLAlchemy and PostgreSQL, access and auditing controls (RBAC and RLS), and deployment using Docker, Vercel, and Neon. The workflow is organized using Agile Scrum, with two-week Sprints tracked in Corestream and version control managed in GitHub.

RECORDATORIO DEL PROYECTO

Problema:

necesidad de centralizar la gestión operativa, los clientes y los indicadores financieros de la plataforma.

Objetivo general:

desarrollar una consola interna con una API segura y una base de datos relacional.

Usuarios:

administradores internos de Acredittia.

Alcance del MVP:

gestión de usuarios y clientes, monitoreo de sistemas, control de incidentes e indicadores financieros.

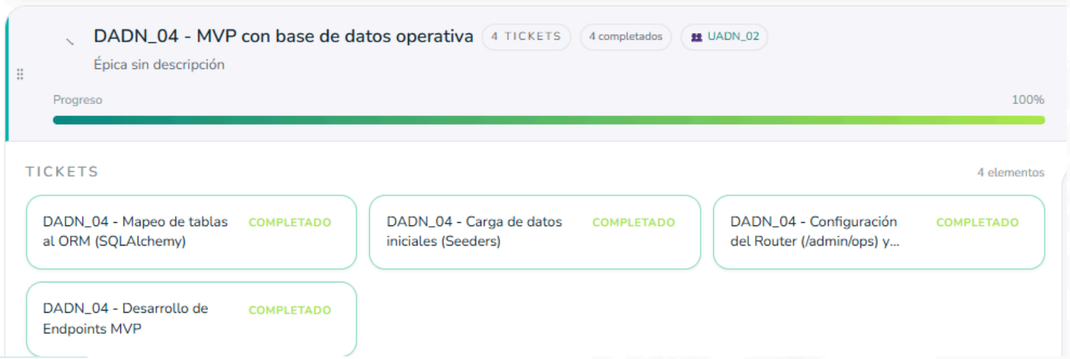
RETROALIMENTACIÓN DE LA FASE 1 Y AJUSTES

Observación o factor	Ajuste realizado	Origen	Justificación
Ajustar precisión técnica y estructura temporal de la Carta Gantt.	Rediseño de la Carta Gantt alineando entregas a los 3 Hitos oficiales.	Retro Docente	Garantizar la trazabilidad y cumplimiento estricto de las rúbricas y fases de evaluación.
Bloqueos de permisos en tablas protegidas por políticas RLS en PostgreSQL.	Inyección del contexto transaccional set_ctx(is_admin=True) en sesiones y seeders	Dificultad	Cumplir la política p_ops_solo_admin garantizando acceso administrativo sin romper RLS.
Conflictos de puertos (8000) y contenedores obsoletos en entorno local.	Depuración de contenedores y estandarización de arquitectura híbrida (Docker para BD/Redis + Uvicorn local)	Dificultad	Permite probar el código actualizado en vivo sin fricción en el caché de contenedores.
Aceleración en el diseño conceptual del esquema relacional.	Adelanto del despliegue cloud en Vercel/Neon y de los primeros 6 endpoints en Hito 1	Facilitador	La disponibilidad del Anexo A del cliente permitió adelantar la arquitectura base.

METODOLOGÍA APLICADA

METODOLOGÍA APLICADA (ÁGIL SCRUM)

- **Sprints Ejecutados: Ciclos iterativos de 2 semanas (Sprints 1 al 5 ejecutados entre Fase 1 y Fase 2), logrando el 100% del Hito 1 (MVP) y el 80% del Hito 2.**
- **Tablero y Backlog (Corestream): Gestión por épicas y tickets técnicos asignados al equipo DADN_04 (Mapeo ORM, Seeders, Router RBAC y Endpoints).**
- **Ceremonias y Control: Daily Stand-ups asíncronas, Sprint Reviews cada miércoles y control de versiones en GitHub mediante ramas y Pull Requests.**
- **Definition of Done (DoD):**
  - **Validación funcional 200 OK / 403 Forbidden en Swagger UI y Postman.**
  - **Cumplimiento de esquemas Pydantic v2 y políticas RLS antes de integrar a main.**

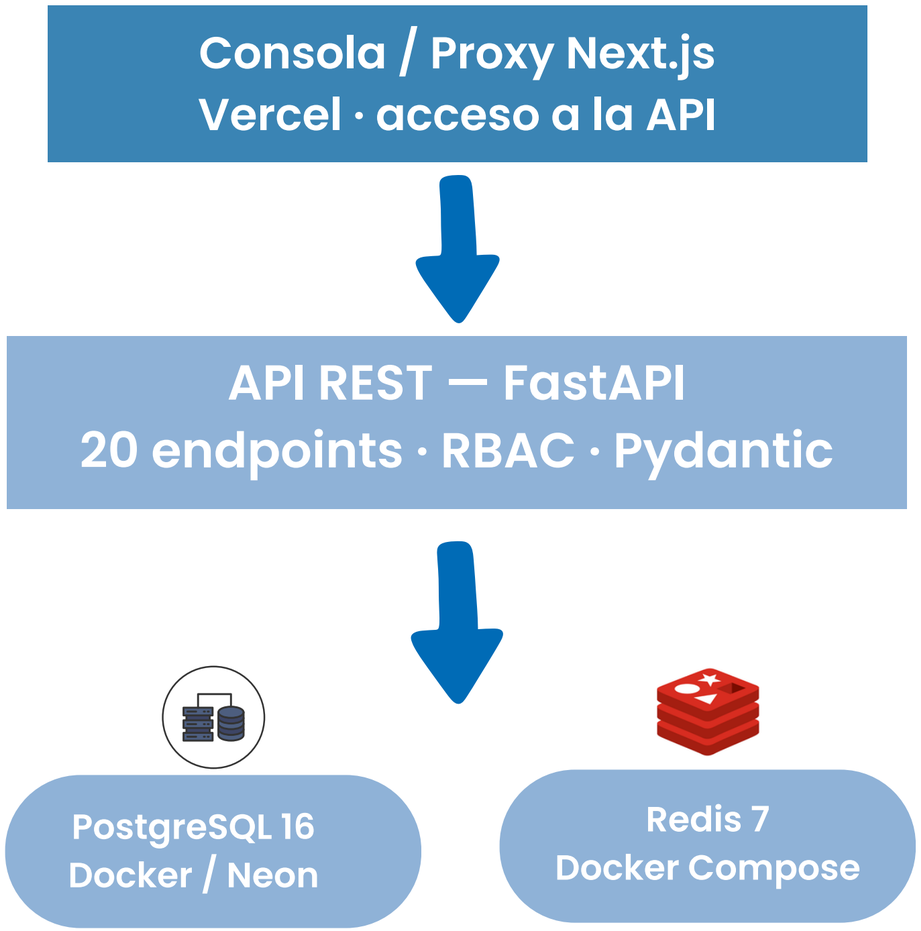


Actividad	Fase 1						
	S1	S2	S3	S4	S5	S6	S7
1. Planificación y Arquitectura Base							
1.1 Levantamiento de requisitos (Equipo)							
1.2 Análisis y definición de arquitectura (Equipo)							
1.3 Repositorio Github y confidencialidad (Angel)							
2. Análisis y Modelamiento de Base de Datos							
2.1 Revisión del esquema SQL (Angel)							
2.2 Implementación de modelos ORM con SQLAlchemy (Angel)							
2.3 Configuración RLS y roles (Angel y Paula)							
2.4 Desarrollo de Seeders idempotentes (Angel)							
2.5 Creación del Job de migraciones (Angel)							
3. Desarrollo de API REST BACKEND							
3.1 Configuración de estructura base FastAPI (Paula)							
3.2 Implementación de Seguridad Perimetral y RBAC (Paula)							
3.3 Desarrollo de endpoints de administración y esquemas Pydantic (Paula)							
3.4 Desarrollo de catálogos globales y métricas operativas (Paula)							

ESTADO DEL PLAN DE TRABAJO

Actividad	Estado	Observación
Planificación y arquitectura	Completado	Arquitectura base definida.
Modelamiento de base de datos	Completado	7 tablas ORM y 6 ENUM.
API REST administrativa	Completado en su alcance operativo	20 endpoints implementados.
Procesos asíncronos	En curso / ajustada	Postergado inicialmente por dependencia con la estabilización de la BD y endpoints del MVP; falta conectar el scheduler automático de Celery Beat
Pasarela de pagos y webhooks	No iniciado	Programado para el Hito 3.
Aseguramiento de calidad	En curso	Falta la batería final de regresión.
Dockerización	En curso	PostgreSQL y Redis operativos en Docker Compose.
Despliegue y documentación	En curso / ajustada	Despliegue cloud adelantado; cierre y actualización pendientes.

ARQUITECTURA ACTUAL



Celery Beat: automatización pendiente de integración.

MODELO DE DATOS IMPLEMENTADO

01. ESQUEMA SQL Y 7 TABLAS ORM IMPLEMENTADAS (MODELS_OPS.PY)

- Núcleo y Clientes: companies (+ extensión dinámica columna industria) y users.
- Monitoreo y Procesos: ops_componentes (PK clave) ——< ops_disponibilidad_diaria (FK componente), ops_jobs (PK clave) ——< ops_job_runs (FK job_clave) y ops_incidentes.
- • Finanzas SaaS: metricas_mensuales y metricas_mensuales_plan (UF / CLP).

02. 6 ENUMS NATIVOS Y SEGURIDAD RLS EN POSTGRESQL 16

- Tipos estrictos: company_industria, ops_job_tipo, ops_run_status, ops_run_trigger, ops_comp_estado, ops_inc_estado.
- • Aislamiento Row-Level Security activo mediante política p_ops_solo_admin.

03. SCRIPTS DE MIGRACIÓN Y SEEDERS IDEMPOTENTES (SEEDS_OPS.PY)

- Inyección transaccional set_ctx(is_admin=True) con validación anti-duplicados (0 colisiones).
- Datos de prueba cargados: 6 componentes, 30 días de disponibilidad (1.440 checks/día), catálogo de jobs e historial de incidentes y finanzas.

```
PS C:\Users\angel\OneDrive\Desktop\Pruebas-main> docker compose exec api python -m app.seeds_ops
>>
>>
2026-10-07 21:32:08,303 [INFO] acredittia.seeds_ops: Iniciando script de seeds para /admin/ops...
2026-10-07 21:32:08,412 [INFO] acredittia.seeds_ops: ¡Seeder ejecutado exitosamente!
2026-10-07 21:32:08,412 [INFO] acredittia.seeds_ops:   * componentes_creados: 0
2026-10-07 21:32:08,412 [INFO] acredittia.seeds_ops:   * jobs_creados: 0
2026-10-07 21:32:08,412 [INFO] acredittia.seeds_ops:   * runs_creados: 0
2026-10-07 21:32:08,412 [INFO] acredittia.seeds_ops:   * incidentes_creados: 0
2026-10-07 21:32:08,412 [INFO] acredittia.seeds_ops:   * disponibilidad_creada: 21
PS C:\Users\angel\OneDrive\Desktop\Pruebas-main>
```

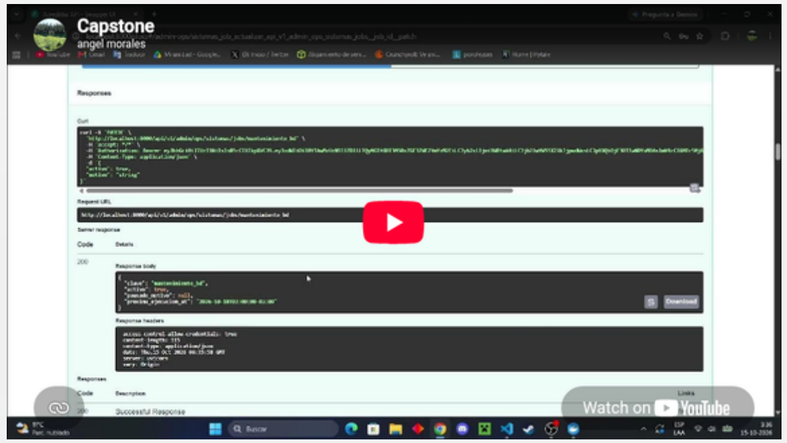
```
SELECT * FROM ops_componentes LIMIT 100
Search Results   Export   Cost: 2ms   1   Total 6
```

clave text	* nombre text	descripcion text	* orden smallint	* activo boolean
db	Base de datos	PostgreSQL 16 · Esquema m	2	true
ia	Verificación IA	Motor OCR y validación aut	5	true
integraciones	Integraciones externas	Sincronización con SIGA, W	6	true
storage	Storage de documentos	Azure Blob Storage / almac	4	true

FUNCIONALIDADES DESARROLLADAS

6	9	5
Endpoints de usuarios y clientes	Endpoints de sistemas, jobs e incidentes	Endpoints financieros

TOTAL: 20 ENDPOINTS IMPLEMENTADOS BAJO /API/V1/ADMIN/OPS.



FUNCIONALIDADES PRINCIPALES:

- CONSULTA Y FILTRADO DE CLIENTES.
- MONITOREO DE COMPONENTES Y DISPONIBILIDAD.
- CONSULTA, EJECUCIÓN Y CONTROL DE JOBS.
- REGISTRO Y RESOLUCIÓN DE INCIDENTES.
- INDICADORES FINANCIEROS COMO MRR, CHURN E INGRESOS POR PLAN.

AVANCE CONTRA EL BACKLOG TOTAL:

- BACKLOG CONSOLA OPERATIVA (/ADMIN/OPS — HITOS 1 Y 2): 20 DE 20 REQUERIMIENTOS/ENDPOINTS COMPLETADOS (100%).
- BACKLOG GLOBAL DEL PROYECTO (FASES CARTA GANTT): 7 DE 9 MÓDULOS COMPLETADOS O EN EJECUCIÓN AVANZADA (78%).

Documentación producida

- **Especificación de Requerimientos / Arquitectura:** Documento base de arquitectura y esquema relacional para consola interna.
- **OpenAPI / Swagger Spec:** Documentación interactiva autogenerada con esquemas Pydantic v2.
- **Backlog del Proyecto:** Registrado y categorizado por tickets en Corestream.
- **Plan de Pruebas:** Colección de Postman y scripts automatizados de verificación RBAC.
- **README del Repositorio:** Documentación de arquitectura, comandos Docker, variables de entorno y guía de ejecución local.

CALIDAD Y ESTÁNDARES

Seguridad Perimetral (RBAC):

Uso del middleware/dependencia require_admin en FastAPI. Rechazo automático con HTTP 403 Forbidden para roles no autorizados.

Estándares de Código y Linter:

- **Tipado estricto con Python type hints y Pydantic v2 para contratos de entrada/salida.**
- **Cumplimiento de convenciones PEP8 mediante Linters en el editor/entorno.**

Estrategia de Git:

Flujo GitFlow estructurado (main, develop, y ramas de feature por ticket) con revisión mediante Pull Requests (PRs).

Aseguramiento QA:

Pruebas funcionales de endpoints con pytest y Swagger UI.

Repositorio Git

ESTRUCTURA DEL REPOSITORIO (BACKEND):

- **/backend/app/models_ops.py: Mapeo ORM en SQLAlchemy 2.0 (7 tablas operativas, 6 ENUMs nativos y extensión Company.industria).**
- **/backend/app/services/ops.py: Router /api/v1/admin/ops con seguridad RBAC (require_admin), esquemas Pydantic v2 y los 20 endpoints (§5 Clientes, §6 Sistemas/Jobs/Incidentes y §7 Finanzas).**
- **/backend/app/seeds_ops.py: Script de poblamiento inicial idempotente compatible con políticas RLS (is_admin=True).**
- **Raíz y Configuración: docker-compose.yml (PostgreSQL 16 y Redis 7), main.py y README.md (arquitectura, variables .env y despliegue).**
- **Ramas: Flujo en main / develop e integración validada en Swagger UI.**

APORTE POR INTEGRANTE

Integrante	Rol	Módulo a Cargo	Tickets / Issues Cerrados	Aporte Técnico y Validación (Commits / PRs)
Angel Morales	Scrum Master y Data Engineer	Organización de sprints, modelamiento ORM y base de datos.	4 / 4 Tickets Hito 1 + Hito 2	Integración centralizada en Git (main), 7 tablas ORM + 6 ENUMs, RLS, seeds_ops.py y contenedores Docker.
Paula Molina	Backend Developer (REST API)	Desarrollo de endpoints y lógica de negocio de la API.	20 / 20 Endpoints Operativos	Programación en sesiones conjuntas (Pair Programming) de rutas FastAPI, esquemas Pydantic y despliegue en Vercel.
Antonia Valencia	Backend Developer y QA	Validación funcional, pruebas de endpoints y apoyo en procesos asíncronos.	Suite QA / 20 Rutas Validadas	Diseño y ejecución de pruebas funcionales (200 OK vs 403), validación de jobs/incidentes y documentación.

RIESGOS Y PLAN AL INFORME FINAL

1. **Aislamiento Multi-tenant y RLS:** Riesgo de fuga de datos cross-tenant. Mitigación: Validación centralizada con require_admin y bypass controlado set_ctx(is_admin=True) en backend.
2. **Sobrecarga en Cola Asíncrona (Celery):** Retrasos en ejecución de jobs periódicos. Mitigación: Monitoreo vía Redis y endpoints de control para pausar/reanudar procesos.
3. I**nconsistencia en Webhooks de Pago:** Fallas de conciliación en Hito 3. Mitigación: Diseño de tabla de logs e idempotencia en recepción de eventos.

RIESGOS Y PLAN AL INFORME FINAL

Compromisos para Semana 15 (Hito 3):

- Integración con pasarela de pago y recepción de Webhooks.
- Activación de Celery Beat para ejecución automática de tareas en segundo plano.
- Cobertura de pruebas automatizadas QA con Pytest y entrega final.

CONCLUSIONES INDIVIDUALES

ANGEL MORALES

Completing Milestone 1 and advancing Milestone 2 confirmed that a robust relational architecture is essential for a SaaS backoffice. Implementing 7 ORM models, 6 PostgreSQL ENUMs, and RLS policies guaranteed data integrity without breaking the core platform. As Scrum Master, organizing two-week Sprints in Corestream was key to minimizing integration risks

CONCLUSIONES INDIVIDUALES

PAULA MOLINA

Designing the REST API layer demonstrated the importance of a fail-closed architecture with strict security. Mounting the /admin/ops router with require_admin dependencies and Pydantic v2 schemas ensured unauthorized requests receive 403 Forbidden, protecting sensitive client and financial metrics across tenants.

CONCLUSIONES INDIVIDUALES

ANTONIA VALENCIA

Enforcing a strict Definition of Done was critical to maintaining project stability. Validating routes via Postman and Swagger UI allowed us to verify complex business rules—such as the 30-day availability series and incident resolution workflows—early in development, preparing our infrastructure for automated background tasks.

CONTACTO:
ANGEL MORALES: AN.MORALESM@DUOCUC.CL
PAULA MOLINA: PAU.MOLINA@DUOCUC.CL
ANTONIA VALENCIA: ANTO.VALENCIA@DUOCUC.CL

Preguntas

REPOSITORIO OFICIAL GITHUB:
HTTPS://GITHUB.COM/ANGELMORALES-1231/CAPSTONE-GRUPO5.GIT